Para validar las rutas, parámetros y payloads de un proyecto Spring Boot utilizando Postman, sigue estos pasos estructurados desde la configuración inicial hasta la automatización de pruebas.

1. Verificación Inicial de la Aplicación

1. Inicia tu aplicación Spring Boot (`mvn spring-boot:run` o desde tu IDE).
2. Confirma el puerto en el que escucha la aplicación (por defecto es `http://localhost:8080`).

2. Estructuración del Proyecto en Postman

1. **Abrir Postman.**
2. Indicar datos iniciales, y en el chat indica “Classical GUI”.
3. **Crear una Colección:**
    - Abre Postman y haz clic en **New** > **Collection**. Nómbrala como tu proyecto (ej. `API Tienda Spring Boot`).
4. **Configurar Variables de Entorno (Recomendado):**
    - En la barra superior derecha, crea un nuevo **Environment** llamado `Local`.
    - Añade una variable:
        - **Variable:** `baseUrl`
        - **Initial/Current Value:** `http://localhost:8080`
    - Activa el entorno `Local`. De esta forma, si cambias de puerto o despliegas a producción, solo tendrás que modificar esta variable.

3. Pruebas por Tipo de Ruta y Parámetro

Añade peticiones a tu colección en Postman según la anotación utilizada en tu controlador Java:

A. Parámetros de Ruta (`@PathVariable`)

Se utilizan para identificar un recurso específico mediante la URL.xº

- **Anotación en Spring Boot:**

```
@GetMapping("/editar/{id}")
public String editar(Model model, @PathVariable int id) { ... }
```

- **Configuración en Postman:**
    - **Método:** `GET`
    - **URL:** `{{baseUrl}}/clientes/editar/:id`

- En la pestaña **Params**, aparecerá automáticamente la sección **Path Variables**. En la columna `Value` del parámetro `id`, escribe el valor numérico (ej. `1`).

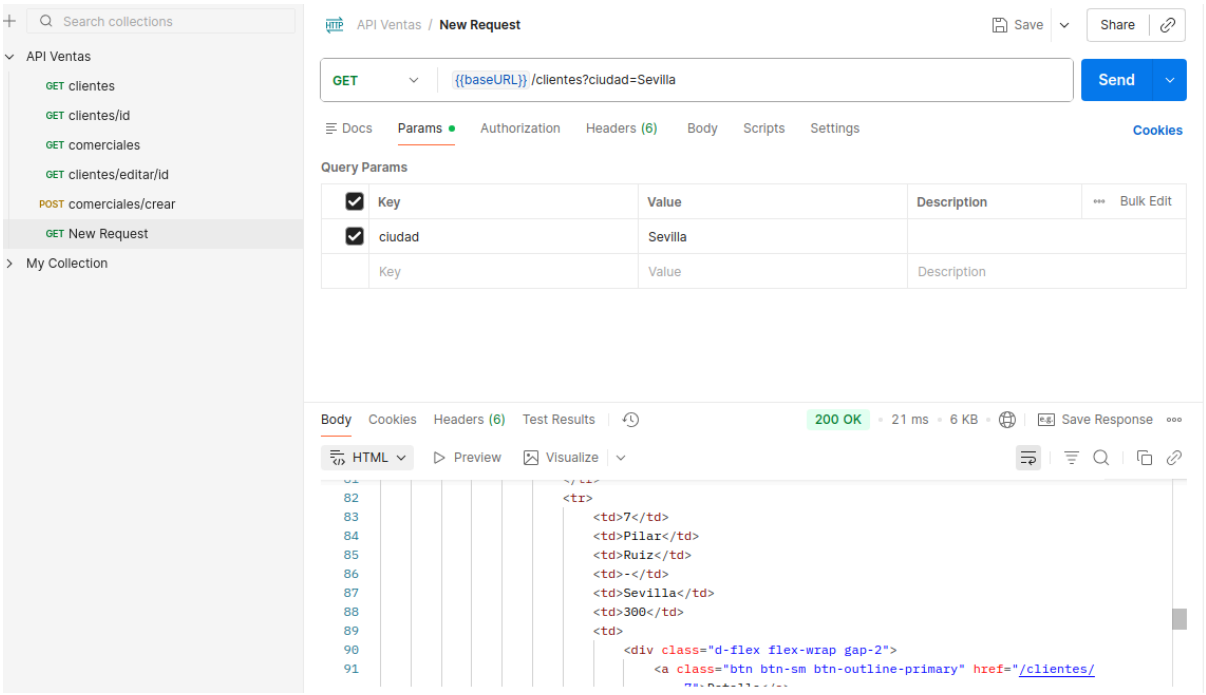
B. Parámetros de Consulta (`@RequestParam`)

Se usan para filtrar, ordenar o paginar resultados.

- **Anotación en Spring Boot:**

@GetMapping("/clientes")
public String listar(@RequestParam(required = false) String ciudad, Model model) { ... }

- **Configuración en Postman:**
  - **Método:** `GET`
  - **URL:** `{{baseUrl}}/clientes`
  - Ve a la pestaña **Params** (Query Params) e introduce las claves y valores:
    - `KEY`: `ciudad` | `VALUE`: `Sevilla`
  - Postman construirá automáticamente la URL: `{{baseUrl}}/clientes?ciudad=Sevilla`.



C. Cuerpo de la Petición (`@RequestBody`)

Se utiliza para enviar datos estructurados en peticiones `POST`, `PUT` o `PATCH`.

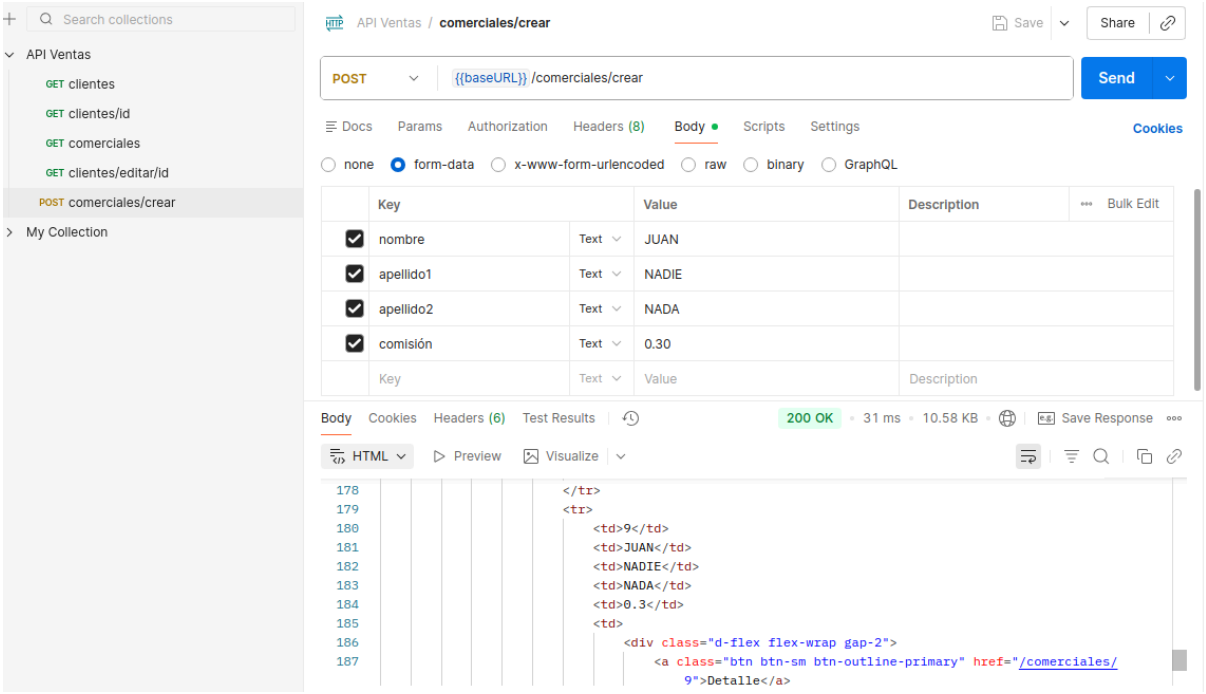
- **Anotación en Spring Boot:**
- Java

@PostMapping("/comerciales/crear")
public RedirectView submitCrear(@ModelAttribute("comercial") Comercial comercial) { ... }

- **Configuración en Postman:**
  - **Método:** `POST`
  - **URL:** `{{baseUrl}}/comerciales/crear`
  - Ve a la pestaña **Body** > Selecciona **form-data** >
  - Escribe en key el nombre de los atributos y en Value los valores a almacenar.

nombre=Juan, apellido1=Nadie, apellido2=Nada

  - En la pestaña **Headers**, Postman añadirá automáticamente `Content-Type: application/json`.



D. Cabeceras (`@RequestHeader`) y Autenticación

Si tu API requiere tokens JWT (Spring Security) o cabeceras personalizadas:

- **Para cabeceras sueltas (`@RequestHeader`):**
  - Ve a la pestaña **Headers** e introduce `KEY` (ej. `X-Tenant-ID`) y `VALUE` (ej. `ES_01`).
- **Para Spring Security (Bearer Token / JWT):**
  - Ve a la pestaña **Authorization** de la petición (o de la colección completa).
  - Selecciona **Type:** `Bearer Token` y pega tu token en el campo `Token`.

4. Automatización de Validaciones (Scripts de Prueba)

Para evitar comprobar las respuestas manualmente cada vez, utiliza la pestaña **Scripts** (o **Tests**) en cada petición para añadir aserciones automáticas en JavaScript:

JavaScript
```
// 1. Validar el código de estado HTTP (ej. 200 OK o 201 Created)
pm.test("Estado de respuesta es 200 OK", function () {
    pm.response.to.have.status(200);
});

// 2. Validar que la respuesta sea un JSON válido
pm.test("El tipo de contenido es JSON", function () {
    pm.response.to.have.header("Content-Type", /application\/json/);
});

// 3. Validar atributos específicos del cuerpo de la respuesta
pm.test("Comprobar datos del producto devuelto", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.nombre).to.eql("Teclado Mecánico");
    pm.expect(jsonData.id).to.be.a('number');
});
```

5. Importación Automática mediante OpenAPI / Swagger

Si utilizas la librería `springdoc-openapi` en tu proyecto Spring Boot, puedes generar automáticamente todas las peticiones en Postman sin crearlas a mano:

1. Añade la dependencia a tu `pom.xml` (si no la tienes):
2. XML

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.3.0</version>
</dependency>
```

3. Arranca la aplicación y verifica la documentación JSON en `http://localhost:8080/v3/api-docs`.
4. En Postman, haz clic en el botón **Import** (esquina superior izquierda).
5. Pega la URL `http://localhost:8080/v3/api-docs` o selecciona **Link**.
6. Postman creará una colección completa con todas las rutas, controladores, parámetros y modelos JSON definidos en tu código Java.